

Laboratório RISC-V Hipotenusa

Boa pergunta! Em Assembly RISC-V, você **chama uma função** (ou subrotina) usando a instrução:

```
assembly

`jal nome_da_funcao
```

✓ O que `jal` faz?

- `jal` significa **Jump And Link**.
- Ela faz duas coisas ao mesmo tempo:
 1. Salta para o **endereço da função** que você quer chamar.
 2. **Salva o endereço de retorno** (a próxima instrução após o `jal`) no registrador `ra` (return address).

Isso permite que, ao final da função, você possa voltar para onde parou usando:

```
assembly

`ret
```

Que, por baixo dos panos, é o mesmo que:

```
assembly

`jalr x0, ra, 0 # volta pro endereço salvo em ra
```

🧠 Exemplo prático

```
assembly

`.globl _start

_start:
    jal main      # chama a função main
    li a0, 0
    li a7, 93     # syscall de exit
    ecall

main:
    jal read_input # chama função de leitura
    jal do_stuff   # faz o processamento
    jal write_out  # escreve a saída
    ret

read_input:
    # faz leitura com syscall 63
    ret
```

```
do_stuff:
    # faz os cálculos da hipotenusa
    ret

write_out:
    # faz saída com syscall 64
    ret
`
```

Dicas

- Sempre que for criar uma função, **coloque um `ret` no final** pra garantir que você volte corretamente.
- Se for usar registradores temporários (`t0`, `t1`, etc.) ou salvar variáveis em `s0`, `s1` ... e fizer chamadas dentro de funções, cuidado com o **convenção de chamada** (calling convention), que define quem deve salvar/restaurar os registradores (posso te explicar isso se quiser depois).
- Pode usar também `jalr` quando quiser pular pra um endereço armazenado num registrador.

Se quiser, posso te ajudar a estruturar suas funções no laboratório agora mesmo. Quer que a gente defina juntos as funções pra parsing, cálculo, e output?